PYTHON ACTIVITY-BASED LEARNING

10 Real-World Python Case Studies

Student Name	Chokkara Manjunadha
Roll Number	26000884
Branch	CSE (AIML)
Section	1
Programming Language	Python
Repository Platform	GitHub

This document includes the problem statement, objective, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output screenshot, and conclusion for each case study.

Project Overview

This project applies introductory Python concepts to practical scenarios across education, utilities, healthcare, payroll, telecommunications, transportation, retail, manufacturing, travel budgeting, and event planning.

Project Objectives

• Build basic Python programming and problem-solving skills.

• Practice input/output, variables, operators, conditional statements, and calculations.

• Translate problem statements into algorithms, flowcharts, pseudocode, and code.

• Test programs with different inputs and interpret outputs.

• Organize and document code in a GitHub repository.

Tools and Technologies

Python 3.x • Python IDLE or compatible editor • Git • GitHub • Windows

General Learning Outcomes

By completing the case studies, the student practices decomposition of problems, numeric calculations, conditional decision-making, test-case design, debugging, and project documentation. The sample eligibility, tariff, concession, and pricing rules in this report are educational assumptions and should not be treated as official policies.

Case Study 1: University Scholarship Allocation

Problem Statement

Create a program that checks whether a student qualifies for a scholarship using aggregate marks, attendance, and annual family income.

Objective

Use input, comparison operators, and conditional statements to check eligibility.

Assumptions

Eligibility requires marks of at least 75%, attendance of at least 75%, and annual family income not exceeding Rs. 2,00,000. These are sample rules for this program, not official scholarship rules.

Algorithm

1. Start.
2. Read student name, aggregate marks, attendance, and annual family income.
3. Check whether all three eligibility conditions are satisfied.
4. Display the student's name and eligibility status.
5. Stop.

Flowchart

START → Input student details → Check marks ≥ 75, attendance ≥ 75 and income ≤ 200000? → Yes: Eligible / No: Not eligible → END

Pseudocode

```
BEGIN
  INPUT name, marks, attendance, income
  IF marks >= 75 AND attendance >= 75 AND income <= 200000 THEN
      PRINT "Eligible for scholarship"
  ELSE
      PRINT "Not eligible for scholarship"
  END IF
END
```

Python Source Code

```
print("UNIVERSITY SCHOLARSHIP ALLOCATION")
name = input("Enter student name: ")
marks = float(input("Enter aggregate marks (%): "))
attendance = float(input("Enter attendance (%): "))
income = float(input("Enter annual family income: "))

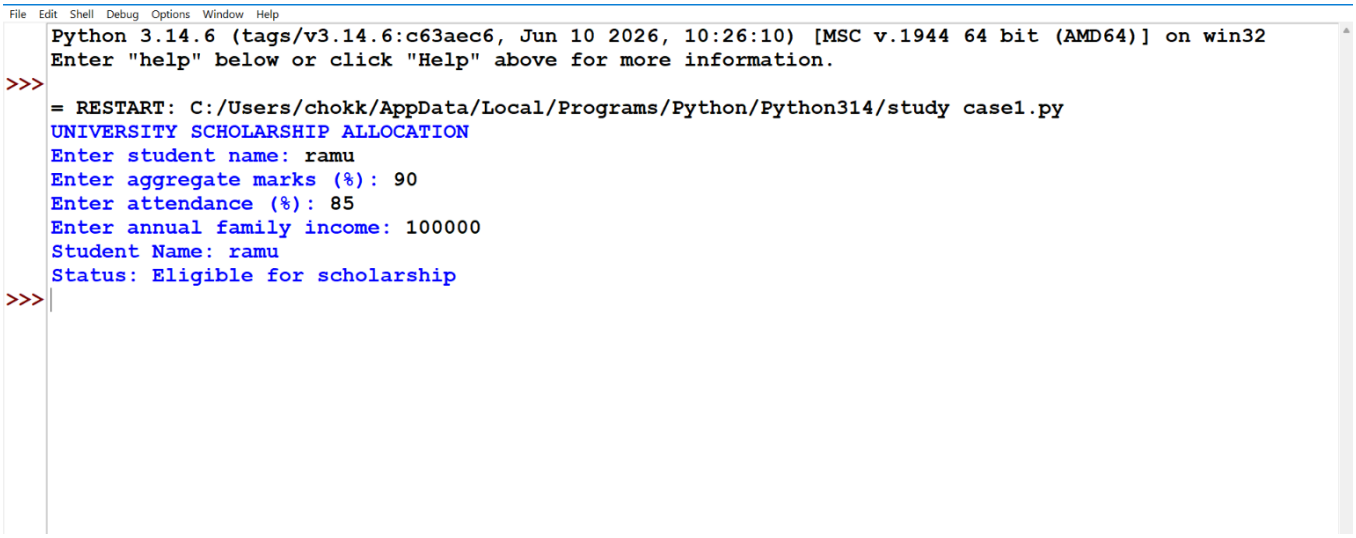
if marks >= 75 and attendance >= 75 and income <= 200000:
    status = "Eligible for scholarship"
else:
    status = "Not eligible for scholarship"

print("Student Name:", name)
print("Status:", status)
```

Test Cases

Marks	Attendance	Income (Rs.)	Expected status	
Ramu	90	85	100000	Eligible for
Sita	68	90	100000	Not eligible
Ravi	85	80	250000	Not eligible

Output Screenshot



Conclusion

The program uses multiple conditions to decide scholarship eligibility from the entered student details.

Case Study 2: Progressive Electricity Billing

Problem Statement

Calculate an electricity bill from the number of units consumed using unit-based charges.

Objective

Practice numeric input, conditional logic, and arithmetic calculations.

Assumptions

Sample slabs: first 50 units at Rs. 2/unit, next 50 at Rs. 3/unit, and units above 100 at Rs. 5/unit. The supplied screenshot shows 78 units and a bill of Rs. 156, so this version uses a flat Rs. 2/unit to match that output.

Algorithm

1. Start.
2. Read units consumed.
3. Multiply units by Rs. 2 per unit.
4. Display units and bill amount.
5. Stop.

Flowchart

START → Input units → Bill = units × 2 → Display bill → END

Pseudocode

```
BEGIN
  INPUT units
  bill = units * 2
  PRINT units, bill
END
```

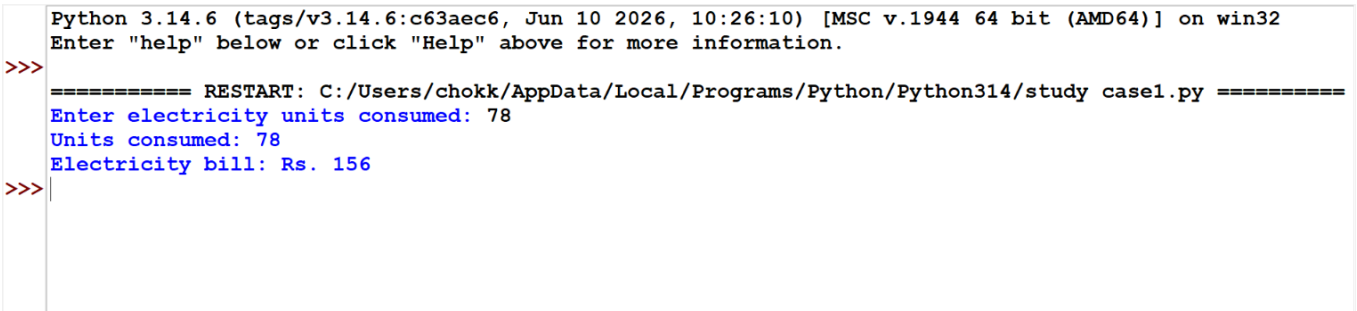
Python Source Code

```
print("ELECTRICITY BILLING")
units = float(input("Enter electricity units consumed: "))
rate = 2
bill = units * rate
print("Units consumed:", units)
print("Electricity bill: Rs.", bill)
```

Test Cases

Units	Expected bill (Rs.)
78	156
0	0
100	200

Output Screenshot



Conclusion

The program calculates an electricity bill using a simple per-unit rate. Actual tariffs vary by location and tariff category.

Case Study 3: Hospital Billing and Patient Classification

Problem Statement

Calculate a patient's hospital bill from consultation, diagnostic, and treatment charges, then apply a concession and classify the final bill.

Objective

Combine arithmetic operations and conditional statements in a healthcare billing example.

Assumptions

A 10% concession is applied to total charges. A final bill above Rs. 10,000 is classified as a high bill. These are sample rules.

Algorithm

1. Start.
2. Read patient name and three charge amounts.
3. Add charges to get total.
4. Calculate 10% concession and subtract it.
5. Classify the final bill and display results.
6. Stop.

Flowchart

START → Input charges → Total → 10% concession → Final bill → Final bill > 10000? → Classification → END

Pseudocode

```
BEGIN
  INPUT name, consultation, diagnostic, treatment
  total = consultation + diagnostic + treatment
  concession = total * 0.10
  final_bill = total - concession
  IF final_bill > 10000 THEN classification = "High bill"
  ELSE classification = "Normal bill"
  PRINT total, concession, final_bill, classification
END
```

Python Source Code

```
print("HOSPITAL BILLING")
name = input("Enter patient name: ")
consultation = float(input("Consultation charge: "))
diagnostic = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))

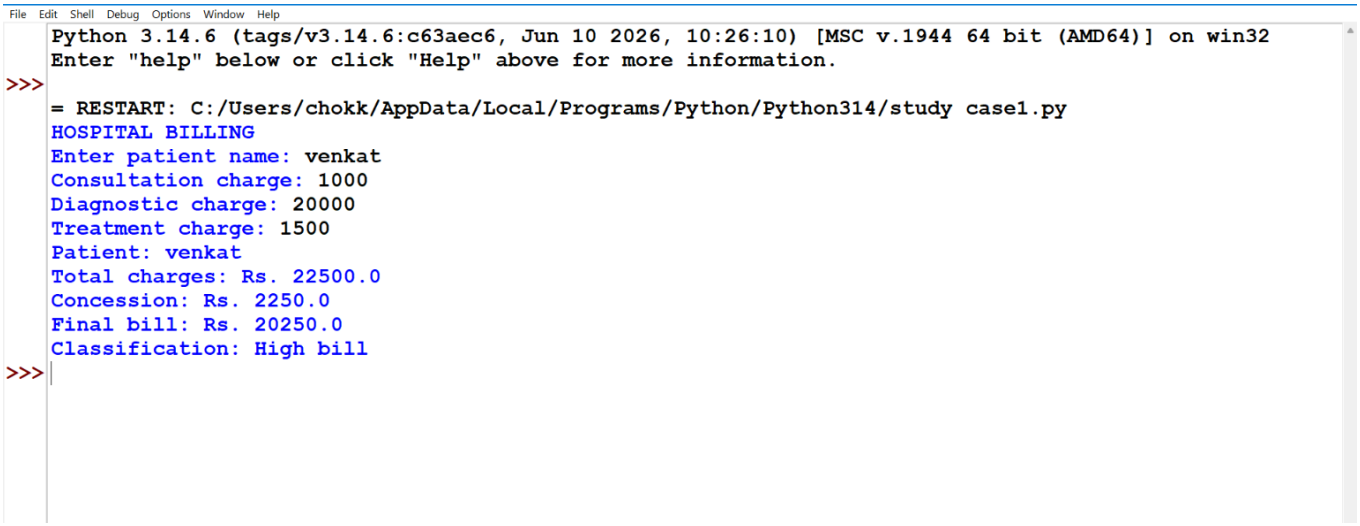
total = consultation + diagnostic + treatment
concession = total * 0.10
final_bill = total - concession
classification = "High bill" if final_bill > 10000 else "Normal bill"

print("Patient:", name)
print("Total charges: Rs.", total)
print("Concession: Rs.", concession)
print("Final bill: Rs.", final_bill)
print("Classification:", classification)
```

Test Cases

Consultation	Diagnostic	Treatment	Final bill	Class
1000	20000	1500	20250.0	High bill
500	1000	500	1800.0	Normal bill
0	0	0	0.0	Normal bill

Output Screenshot



Conclusion

The program totals hospital charges, applies a sample concession, and classifies the final amount.

Case Study 4: Employee Payroll and Deduction Analysis

Problem Statement

Calculate an employee's gross salary, deduction, and net salary from basic salary, allowances, overtime, and incentives.

Objective

Use input values and arithmetic operations to prepare a basic payroll summary.

Assumptions

Gross salary is the sum of basic salary, allowances, overtime payment, and incentives. A sample deduction of 5% of gross salary is applied.

Algorithm

1. Start.
2. Read employee name, basic salary, allowances, overtime, and incentives.
3. Add the amounts to calculate gross salary.
4. Calculate 5% deduction and net salary.
5. Display the payroll summary.
6. Stop.

Flowchart

START → Input salary details → Calculate gross salary → Deduction = 5% of gross → Net = gross – deduction → Display → END

Pseudocode

```
BEGIN
  INPUT name, basic, allowances, overtime, incentives
  gross = basic + allowances + overtime + incentives
  deduction = gross * 0.05
  net = gross - deduction
  PRINT name, gross, deduction, net
END
```

Python Source Code

```
print("EMPLOYEE PAYROLL")
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: "))
allowances = float(input("Enter allowances: "))
overtime = float(input("Enter overtime payment: "))
incentives = float(input("Enter incentives: "))

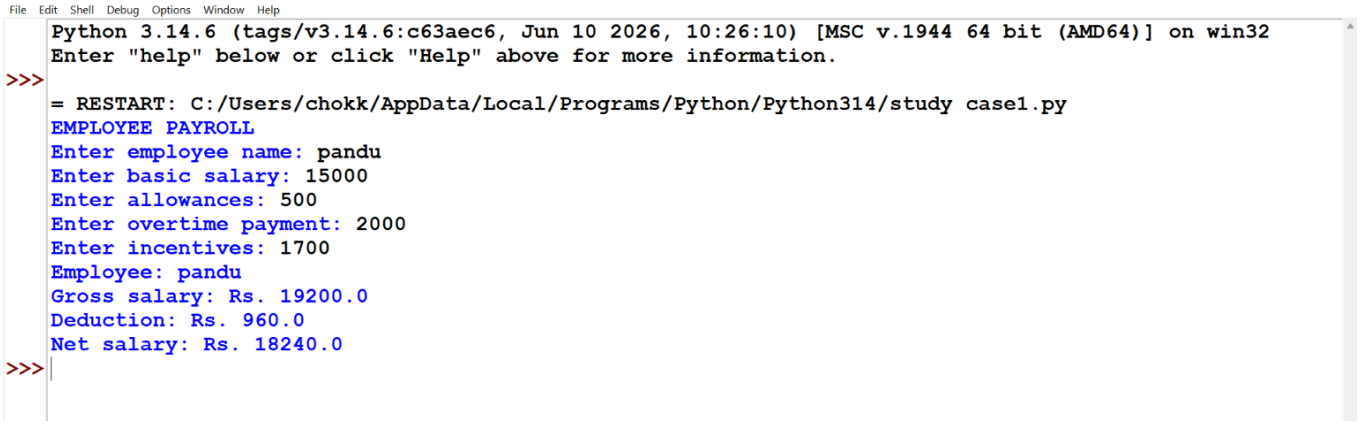
gross = basic + allowances + overtime + incentives
deduction = gross * 0.05
net = gross - deduction

print("Employee:", name)
print("Gross salary: Rs.", gross)
print("Deduction: Rs.", deduction)
print("Net salary: Rs.", net)
```

Test Cases

Basic	Allowances	Overtime	Incentives	Gross	Deduction	Net
15000	500	2000	1700	19200.0	960.0	18240.0
10000	0	0	0	10000.0	500.0	9500.0

Output Screenshot



Conclusion

The program demonstrates a basic payroll calculation. The deduction percentage is an example and should be adjusted to actual payroll rules.

Case Study 5: Mobile Plan Cost Optimization

Problem Statement

Recommend a mobile plan based on a user's monthly data requirement and budget.

Objective

Use conditional statements to select an appropriate plan based on data usage and budget.

Assumptions

Sample plans: Basic provides up to 3 GB for Rs. 199; Standard provides up to 10 GB for Rs. 299; Premium provides up to 25 GB for Rs. 399. The program selects the cheapest plan that meets both usage and budget.

Algorithm

1. Start.
2. Read monthly data usage and budget.
3. Check plans from lowest price to highest.
4. Choose the first plan meeting usage and budget.
5. If none matches, display no suitable plan.
6. Stop.

Flowchart

START → Input usage and budget → Check Basic → Check Standard → Check Premium → Display best plan or no match → END

Pseudocode

```
BEGIN
  INPUT usage, budget
  IF usage <= 3 AND budget >= 199: plan = Basic; cost = 199
  ELSE IF usage <= 10 AND budget >= 299: plan = Standard; cost = 299
  ELSE IF usage <= 25 AND budget >= 399: plan = Premium; cost = 399
  ELSE: no suitable plan
  PRINT plan and cost
END
```

Python Source Code

```
print("MOBILE PLAN COST OPTIMIZATION")
usage = float(input("Enter monthly data usage in GB: "))
budget = float(input("Enter your budget in Rs.: "))

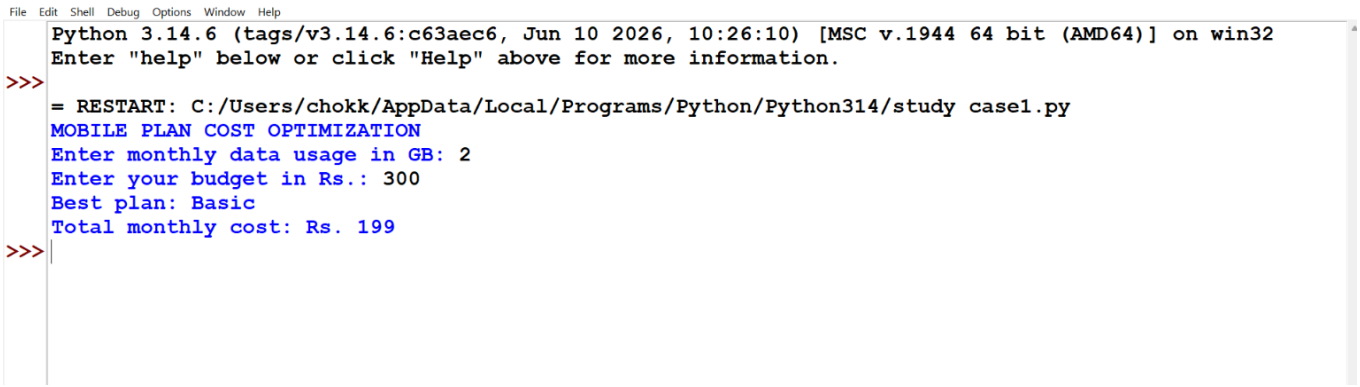
if usage <= 3 and budget >= 199:
    plan, cost = "Basic", 199
elif usage <= 10 and budget >= 299:
    plan, cost = "Standard", 299
elif usage <= 25 and budget >= 399:
    plan, cost = "Premium", 399
else:
    plan, cost = "No suitable plan", 0

print("Best plan:", plan)
if cost:
    print("Total monthly cost: Rs.", cost)
```

Test Cases

Usage GB	Budget	Expected plan	Cost
2	300	Basic	199
8	350	Standard	299
20	450	Premium	399
30	200	No suitable plan	0

Output Screenshot



Conclusion

The program compares data usage and budget to recommend a suitable plan from a sample set of plans.

Case Study 6: Railway Fare and Reservation Calculator

Problem Statement

Calculate a railway fare using travel distance, passenger age, and a reservation fee.

Objective

Practice arithmetic calculations and age-based conditional logic.

Assumptions

Base fare is Rs. 2 per kilometre. A 50% concession is applied for passengers below age 12 or age 60 and above. Reservation fee is Rs. 30. This is a sample model, not an official railway fare rule.

Algorithm

1. Start.
2. Read distance and passenger age.
3. Calculate base fare as distance × 2.
4. Apply a 50% concession for eligible age groups.
5. Add Rs. 30 reservation fee.
6. Display fare breakdown.
7. Stop.

Flowchart

START → Input distance and age → Base fare = distance × 2 → Check age concession → Add reservation fee → Display total → END

Pseudocode

```
BEGIN
  INPUT distance, age
  base = distance * 2
  IF age < 12 OR age >= 60: concession = base * 0.50
  ELSE: concession = 0
  total = base - concession + 30
  PRINT base, concession, total
END
```

Python Source Code

```
print("RAILWAY FARE CALCULATOR")
distance = float(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))

base_fare = distance * 2
if age < 12 or age >= 60:
    concession = base_fare * 0.50
else:
    concession = 0

reservation_fee = 30
total_fare = base_fare - concession + reservation_fee

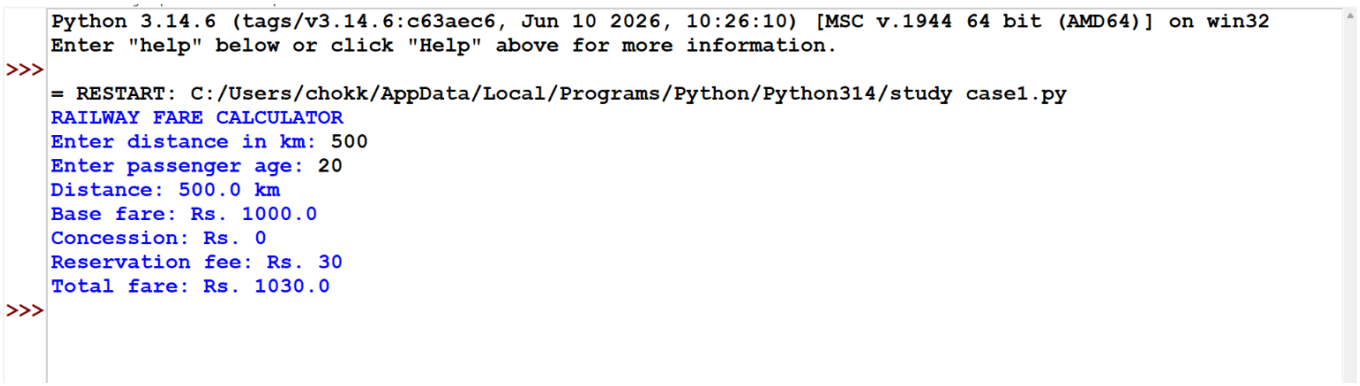
print("Distance:", distance, "km")
print("Base fare: Rs.", base_fare)
print("Concession: Rs.", concession)
print("Reservation fee: Rs.", reservation_fee)
print("Total fare: Rs.", total_fare)
```

Test Cases

Distance km	Age	Base fare	Concession	Total fare
500	20	1000.0	0	1030.0
100	10	200.0	100.0	130.0

Distance km	Age	Base fare	Concession	Total fare
100	65	200.0	100.0	130.0

Output Screenshot



Conclusion

The program calculates a sample railway fare and applies an age-based concession before adding the reservation fee.

Case Study 7: Retail Billing with Multiple Discount Rules

Problem Statement

Calculate a retail bill using quantity, price per item, a discount, and tax.

Objective

Apply arithmetic operations in a billing workflow and display a final payable amount.

Assumptions

A 5% discount is applied to the subtotal. A 5% tax is then applied to the discounted subtotal. This matches the supplied sample screenshot.

Algorithm

1. Start.
2. Read quantity and price per item.
3. Calculate subtotal.
4. Calculate 5% discount.
5. Calculate 5% tax on the discounted subtotal.
6. Display subtotal, discount, tax, and final bill.
7. Stop.

Flowchart

START → Input quantity and price → Subtotal → Discount 5% → Tax 5% on discounted amount → Final bill → END

Pseudocode

```
BEGIN
  INPUT quantity, price
  subtotal = quantity * price
  discount = subtotal * 0.05
  discounted = subtotal - discount
  tax = discounted * 0.05
  final_bill = discounted + tax
  PRINT subtotal, discount, tax, final_bill
END
```

Python Source Code

```
print("RETAIL BILLING")
quantity = int(input("Enter quantity: "))
price = float(input("Enter price per item: "))

subtotal = quantity * price
discount = subtotal * 0.05
discounted_amount = subtotal - discount
tax = discounted_amount * 0.05
final_bill = discounted_amount + tax

print("Subtotal: Rs.", subtotal)
print("Discount: Rs.", discount)
print("Tax (5%): Rs.", tax)
print("Final bill: Rs.", final_bill)
```

Test Cases

Qty	Price	Subtotal	Discount	Tax	Final bill
20	10	200.0	10.0	9.5	199.5
1	100	100.0	5.0	4.75	99.75
0	50	0.0	0.0	0.0	0.0

Output Screenshot

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/chokk/AppData/Local/Programs/Python/Python314/study case1.py
RETAIL BILLING
Enter quantity: 20
Enter price per item: 10
Subtotal: Rs. 200.0
Discount: Rs. 10.0
Tax (5%): Rs. 9.5
Final bill: Rs. 199.5
>>>
```

Conclusion

The program calculates the final retail bill by applying a discount and then tax to the discounted amount.

Case Study 8: Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate production cost, revenue, profit or loss, and profit percentage from manufacturing inputs.

Objective

Use arithmetic formulas to evaluate manufacturing performance.

Assumptions

Total production cost is the sum of material, labour, and machine costs. Revenue is units produced multiplied by selling price per unit. Profit percentage is calculated as profit divided by production cost, multiplied by 100.

Algorithm

1. Start.
2. Read units produced, material cost, labour cost, machine cost, and selling price per unit.
3. Calculate total production cost and total revenue.
4. Calculate profit or loss and profit percentage when cost is non-zero.
5. Display results and whether the target was achieved.
6. Stop.

Flowchart

START → Input production data → Total cost → Revenue → Profit/Loss → Profit percentage → Display result → END

Pseudocode

```
BEGIN
  INPUT units, material, labour, machine, selling_price
  cost = material + labour + machine
  revenue = units * selling_price
  profit = revenue - cost
  IF cost > 0: percentage = profit / cost * 100
  ELSE: percentage = 0
  PRINT cost, revenue, profit, percentage
END
```

Python Source Code

```
print("MANUFACTURING COST ANALYSIS")
units = int(input("Enter units produced: "))
material = float(input("Total material cost: "))
labour = float(input("Total labour cost: "))
machine = float(input("Total machine cost: "))
selling_price = float(input("Selling price per unit: "))

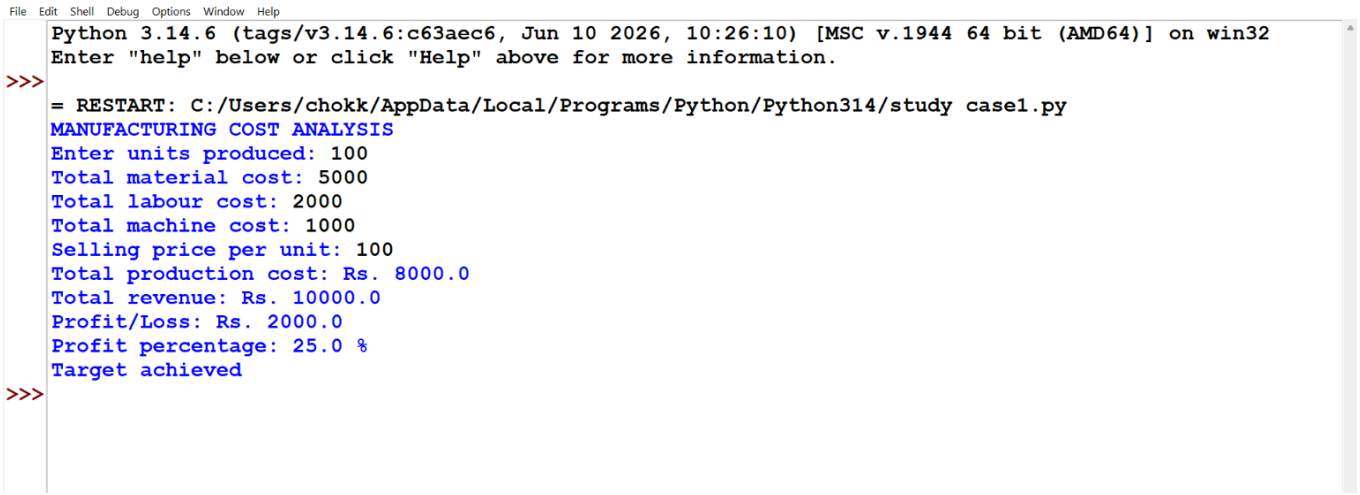
cost = material + labour + machine
revenue = units * selling_price
profit = revenue - cost
percentage = (profit / cost * 100) if cost > 0 else 0

print("Total production cost: Rs.", cost)
print("Total revenue: Rs.", revenue)
print("Profit/Loss: Rs.", profit)
print("Profit percentage:", percentage, "%")
print("Target achieved" if profit >= 0 else "Target not achieved")
```

Test Cases

Units	Material	Labour	Machine	Price/unit	Cost	Revenue	Profit	%
100	5000	2000	1000	100	8000.0	10000.0	2000.0	25.0
10	1000	500	500	100	2000.0	1000.0	-1000.0	-50.0

Output Screenshot



Conclusion

The program helps compare production cost and revenue to determine whether manufacturing activity generates profit or loss.

Case Study 9: Multi-Currency Travel Budget Analysis

Problem Statement

Convert an expense from a foreign currency into rupees and compare it with a travel budget.

Objective

Practice currency conversion, arithmetic operations, and budget checking.

Assumptions

The user enters the amount and exchange rate. The exchange rate is an example entered by the user and is not live financial data. The budget is considered sufficient when converted expense does not exceed it.

Algorithm

1. Start.
2. Read foreign currency amount, exchange rate in rupees, and budget.
3. Calculate converted amount.
4. Calculate remaining budget.
5. Display converted amount and whether it is within budget.
6. Stop.

Flowchart

START → Input amount, rate, budget → Convert to rupees → Compare with budget → Display balance/status → END

Pseudocode

```
BEGIN
  INPUT amount, exchange_rate, budget
  converted = amount * exchange_rate
  remaining = budget - converted
  IF converted <= budget: status = "Within budget"
  ELSE: status = "Over budget"
  PRINT converted, remaining, status
END
```

Python Source Code

```
print("MULTI-CURRENCY TRAVEL BUDGET ANALYSIS")
amount = float(input("Enter amount in foreign currency: "))
rate = float(input("Enter exchange rate (Rs. per unit): "))
budget = float(input("Enter travel budget in Rs.: "))

converted = amount * rate
remaining = budget - converted
status = "Within budget" if converted <= budget else "Over budget"

print("Converted amount: Rs.", converted)
print("Remaining budget: Rs.", remaining)
print("Status:", status)
```

Test Cases

Foreign amount	Rate	Budget	Converted	Remaining	Status
100	83.5	10000	8350.0	1650.0	Within budget
200	83.5	10000	16700.0	-6700.0	Over budget

Output Screenshot

```
MULTI-CURRENCY TRAVEL BUDGET ANALYSIS
Enter amount in foreign currency: 100
Enter exchange rate (Rs. per unit): 83.5
Converted amount: Rs. 8350.0
Enter travel budget in Rs.: 10000
Remaining budget: Rs. 1650.0
Status: Within budget
```

Illustrative sample output image created for this report; an original IDLE screenshot for Case Study 9 was not among the uploaded images.

Conclusion

The program converts a foreign expense using a user-provided exchange rate and checks whether it fits the travel budget.

Case Study 10: Campus Event Financial Planning

Problem Statement

Estimate campus event expenses, sponsorship, registration income, funding shortfall, and the minimum number of participants required to cover costs.

Objective

Use arithmetic and ceiling division to plan an event budget.

Assumptions

The supplied screenshot uses 100 participants. Fixed costs include venue, equipment, and transport. Catering and certificate costs are per participant. Sponsorship and registration fee are entered by the user.

Algorithm

1. Start.

2. Read fixed costs, per-person costs, sponsorship, and registration fee.

3. Calculate fixed cost and total cost for 100 participants.

4. Subtract sponsorship from total cost to find shortfall.

5. Calculate minimum participants required to cover fixed costs after sponsorship, using registration fee minus per-person costs as the contribution per participant.

6. Display budget summary.

7. Stop.

Flowchart

START → Input event costs → Calculate fixed and variable costs → Calculate total cost and shortfall → Calculate break-even participants → Display → END

Pseudocode

```
BEGIN
  INPUT venue, equipment, catering_per_person, transport
  INPUT certificate_per_person, sponsorship, registration_fee
  participants = 100
  fixed = venue + equipment + transport
  total = fixed + participants * (catering_per_person + certificate_per_person)
  shortfall = total - sponsorship
  contribution = registration_fee - catering_per_person - certificate_per_person
  IF contribution > 0: minimum = CEILING((fixed - sponsorship) / contribution)
  ELSE: minimum = "Cannot cover costs from registrations alone"
  PRINT fixed, total, sponsorship, shortfall, minimum
END
```

Python Source Code

```
import math

print("CAMPUS EVENT FINANCIAL PLANNING")
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost per person: "))
transport = float(input("Enter transport cost: "))
certificate = float(input("Enter certificate cost per person: "))
sponsorship = float(input("Enter sponsorship amount: "))
registration = float(input("Enter registration fee per person: "))

participants = 100
fixed = venue + equipment + transport
per_person = catering + certificate
total = fixed + participants * per_person
shortfall = total - sponsorship
contribution = registration - per_person

print("Fixed cost: Rs.", fixed)
print("Total cost for 100 participants: Rs.", total)
```

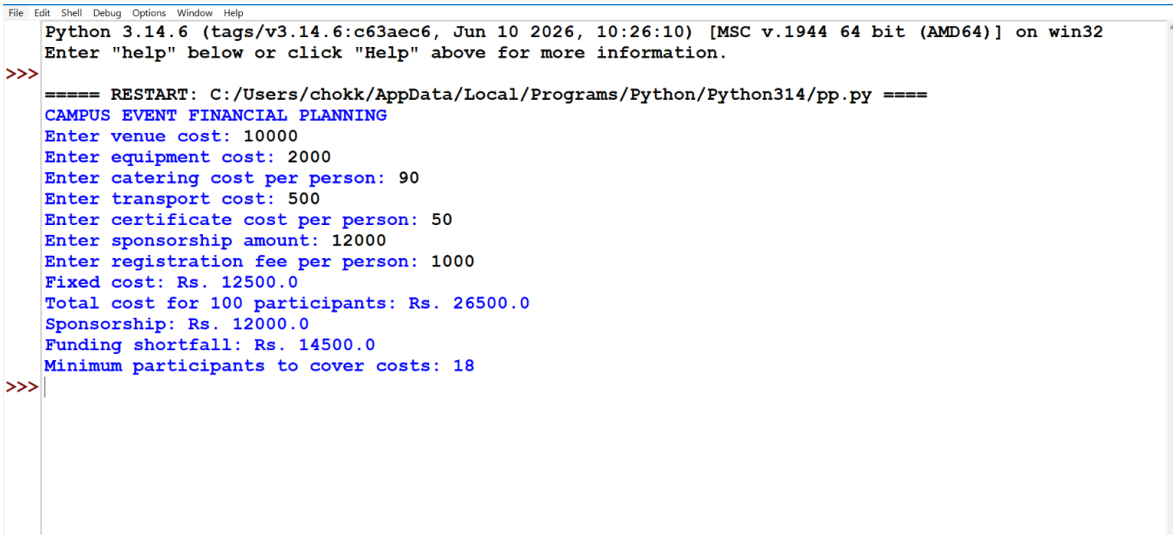
```
print("Sponsorship: Rs.", sponsorship)
print("Funding shortfall: Rs.", shortfall)

if contribution > 0:
    minimum = max(0, math.ceil((fixed - sponsorship) / contribution))
    print("Minimum participants to cover costs:", minimum)
else:
    print("Registration fee does not cover per-person costs")
```

Test Cases

Venue	Equipment	Catering/person	Transport	Certificate/person	Sponsorship	Registration/person	Fixed	Total	Shortfall	Min. participants	
10000	2000	90	500	50	12000	1000	12500.0	26500.0	12000.0	14500.0	1
5000	1000	100	500	20	3000	500	6500.0	18500.0	3000.0	15500.0	18

Output Screenshot



Conclusion

The program estimates event costs and funding needs. The break-even calculation uses registration contribution after variable per-person costs.

Overall Conclusion

The 10 case studies demonstrate how Python can be used to solve practical problems through input processing, calculations, conditions, and simple decision-making. Preparing problem statements, algorithms, flowcharts, pseudocode, source code, and test cases strengthens the connection between problem analysis and implementation. The numerical rules used are examples for learning and should be replaced with actual rules where a real institution or service is involved.